데이터분석기법

14주차: Decision tree algorithms

양승진
slowmoyang@gmail.com

세종대학교

2025년 6월 2일

Decision Tree

- A **decision tree** is a ML model composed of a collection of "questions" organized hierarchically in the shape of an (inverted) tree
- The questions are usually called a condition or a split
- The **root node** is the entry point of the decision tree where the splitting process begins
- Each non-leaf node contains a condition based on one of the input features, which is used to split the data into two branches
- Each **leaf node** make a prediction – either a class label for classification or a numerical value for regression – representing the model's output for inputs that reach that leaf

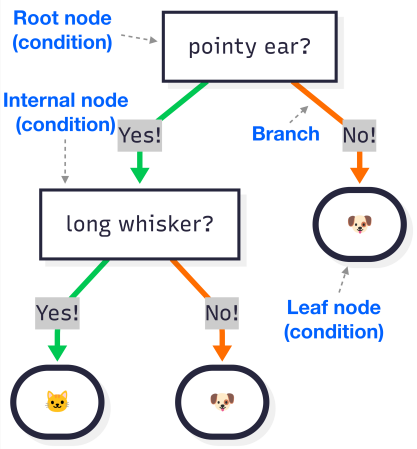


Figure 1

Training a Decision Tree

- Training a DT usually follows a greedy divide-and-conquer approach
- Training begins by making the root node and then builds the tree step by step
- At each node, the algorithm give **scores** to all possible **conditions** and chooses the condition with the best score
  - For example, features are ear pointiness and the average length of whiskers
  - condition = ear pointiness > 0.5
- The algorithm then repeats recursively and independently on both children nodes
- If no suitable condition is found, the node becomes a leaf

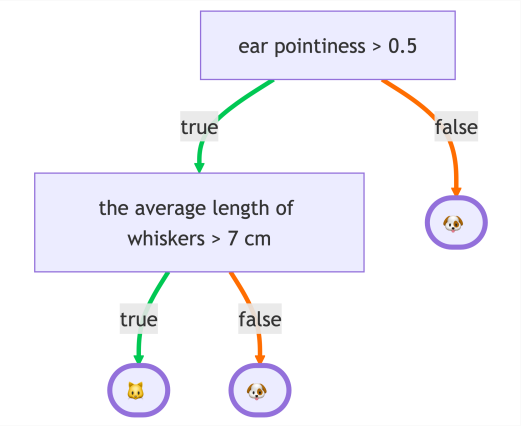


Figure 2

Scores for Classification

- Assumption: binary classification with real-valued features
- The goal is to find a feature x_i and a threshold t such that splitting the training set into two groups based on the condition $x_i \geq t$ improves label separation
- The training algorithm scans possible conditions based on task-specific scores to find the best split at each node

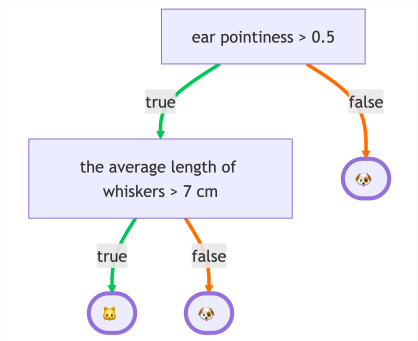
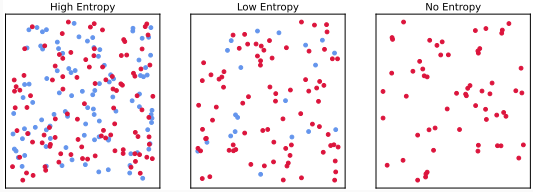


Figure 3

Shannon entropy

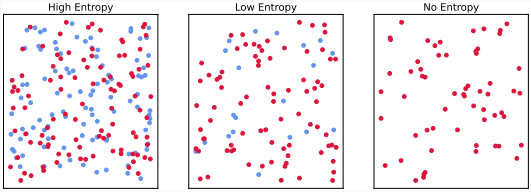


- Shannon entropy is a measure of disorder

$$H(x) = -\sum_i p(x_i) \log p(x_i) \tag{1}$$

- High entropy indicates x is sampled from a flat distribution
- Low entropy indicates x is sampled from a peaky distribution

Information Gain



- Information gain (IG) is a measure of reduction in uncertainty (entropy) after splitting a dataset based on a feature

$$IG = H(\mathrm{P}) - \frac{|L|}{|P|}H(L) - \frac{|R|}{|P|}H(R), \tag{2}$$

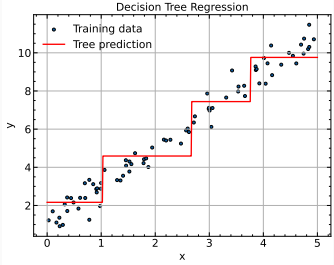
  - P: a parent node, L(R): a left(right) node
  - $|X|$: the number of examples in a node X
- The higher the IG, the better the feature for splitting!
- IG helps the tree decide which feature to split on at each node, and tell how useful a feature is for classification

Decision Tree Regression

- A decision tree can also perform regression
- In regression trees, each leaf node predicts a continuous value, usually the mean of the target values in the corresponding subset
- The commonly used score for regression trees is Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - t_i)^2, \qquad (3)$$

- N: the number of training examples in the node
- t_i: the actual target value of the i-th training example in the node
- y is the predicted value, usually the mean of all t_i values in the node



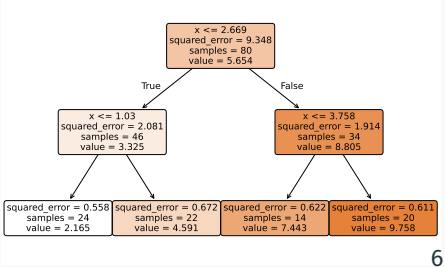
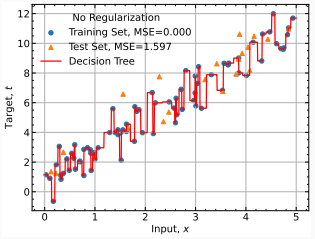


Figure 4

Regularization

- DTs can perfectly fit noisy training data but generalize poorly
- To prevent overfitting in a DT, apply one or both of the following regularization during training



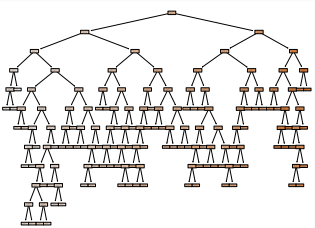
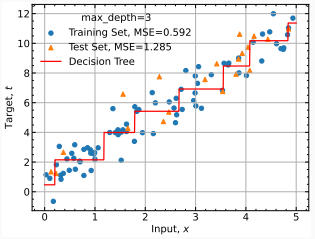


Figure 5: Example of an overfitted decision tree on a linear regression problem

Regularization

- DTs can perfectly fit noisy training data but generalize poorly
- To prevent overfitting in a DT, apply one or both of the following regularization during training
  - Limiting the maximum depth



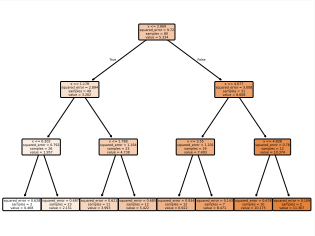


Figure 5: Example of limiting the maximum depth of a decision tree

Regularization

- DTs can perfectly fit noisy training data but generalize poorly
- To prevent overfitting in a DT, apply one or both of the following regularization during training
  - Limiting the maximum depth
  - Limiting the minimum number of examples in each leaf

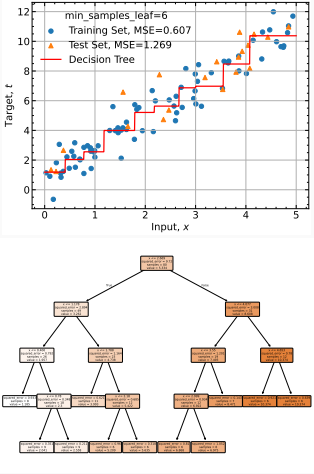
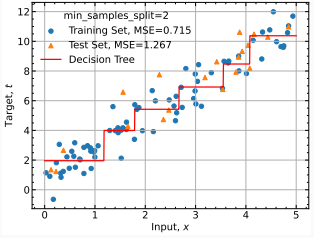


Figure 5: Example of limiting the minimum number of samples in a leaf

Regularization

- DTs can perfectly fit noisy training data but generalize poorly
- To prevent overfitting in a DT, apply one or both of the following regularization during training
  - Limiting the maximum depth
  - Limiting the minimum number of examples in each leaf
- Regularization can also be performed after training by pruning–removing certain branches and converting some non-leaf nodes into leaves



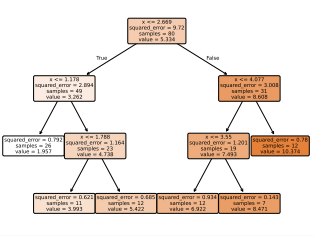


Figure 5: Example of pruning a decision tree

Feature Importance

- Feature importance is a score that indicates how much a feature contributes to the DT
- Permutation feature importance is a method for measuring feature importance by assessing how much a model's prediction error increases when the values of a feature are randomly permuted

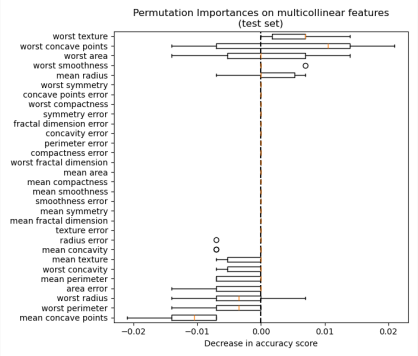


Figure 6: Example of permutation feature importance for a model trained on the Breast Cancer Wisconsin (Diagnostic) dataset. Taken from [1]

Decision Forest

- An ensemble is **a collection of models trained independently,** whose predictions are combined–typically through averaging or voting
- In many cases ensembles outperform individual models
- A decision forest is a ensemble of multiple decision trees

Bagging

- Bagging (bootstrap aggregating) is an ensemble method that trains multiple models in parallel on sampled subsets of the data and combines their predictions through averaging or voting
- A bagged decision tree is an ensemble of decision trees, in which each decision tree is trained on different bootstrapped subsets of the training set

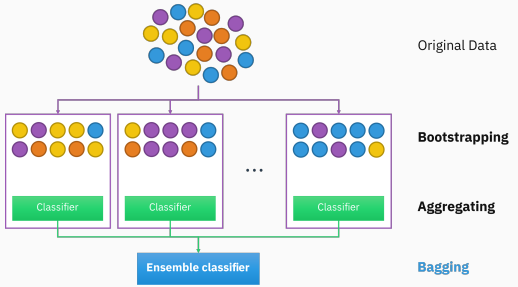


Figure 7: Illustration of the bagging process. Adapted from [3]).

Random Forest

- Random forest = decision trees + bagging + feature bagging
- Feature bagging is a technique where a random subset of features is selected when training a model, reducing the correlation between decision trees
  - If decision trees are high-variance models, small changes in data can lead to very different trees
  - If all trees use the same features, they may become too similar – reducing the benefit of ensembling
  - If decision trees make uncorrelated errors, averaging their predictions can significantly reduce overall variance

Boosting

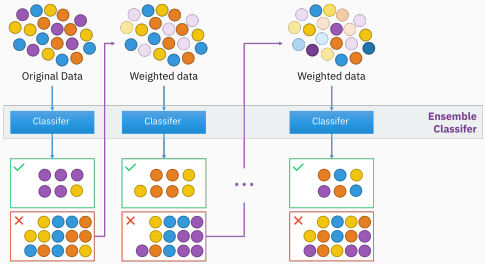


Figure 8: Illustration of the boosting process. Adapted from [2]

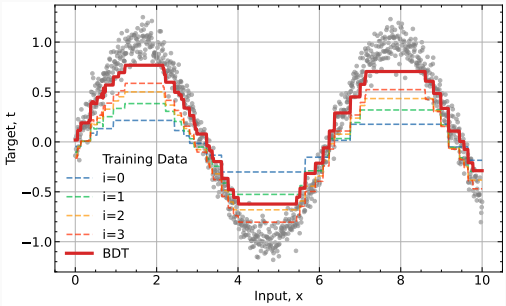


Figure 9: Illustration of a boosted decision tree

- **Boosting** is an ensemble technique that builds a strong ML model by sequentially combining weak ML models, giving large weights on examples that were misclassified by previous models
- Boosted decision tree (BDT) = boosting + decision tree
- Examples: adaptive boosting and gradient boosting

Gradient Boosted Decision Trees

- A **Gradient Boosted Decision Tree (GBDT)** is a type of BDTs to reduce the errors of the previous trees by minimizing a loss function using gradient
- Define a loss function to measure how wrong a prediction is
  - regression: $\mathcal{L}(y,t) = (y-t)^2$, where $y = f(x)$ and f indicates a decision tree and
  - classification: $\mathcal{L}(y,t) = -[t\log(y) + (1-t)\log(1-y)]$, where $y = \sigma(f(x))$ and σ is sigmoid function
- Instead of learning to directly predict the label, **the weak learner predicts the gradient** (how to improve the current prediction)
  - regression: $\frac{\partial L(y_i,t)}{\partial y_i} = \frac{\partial}{\partial y_i}(y_i-t)^2 = 2(y_i-t)$
  - classification: $\frac{\partial \mathcal{L}}{\partial f} = y - t$
- Update the GBDT: $F_{i+1} = F_i - \eta f_i$,
  - F_i and f_i indicates the GBDT and a decision tree at step i
  - η is shrinkage that controls the pace at which the overall model learns, helping to prevent overfitting
- Gradient boosting trains a model to predict gradients, while gradient descent directly updates weights using gradients
- GBDTs are one of the most powerful and widely used machine learning models

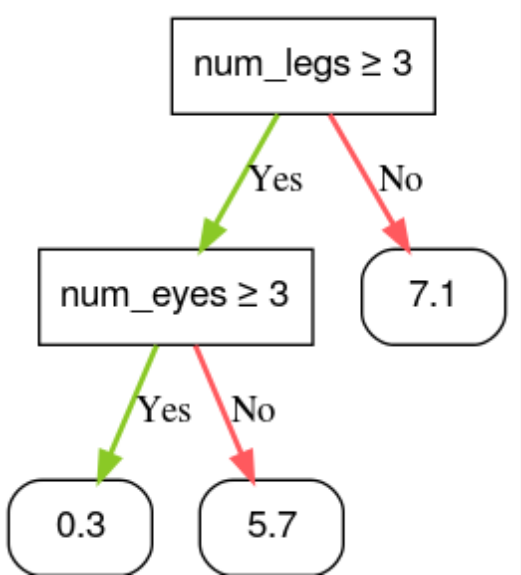
Summary

- Decision Trees are intuitive, flexible models that split data using feature-based conditions
- They can handle both classification and regression tasks
- Trees are built greedily by selecting the best feature and threshold at each node
- Without regularization, decision trees overfit easily $\rightarrow$ Use depth limits, min samples per leaf, or post-pruning to generalize better
- Ensemble Learning boosts performance by combining many trees:
  - Bagging (e.g., Random Forest): reduces variance by averaging over uncorrelated trees
  - Boosting(e.g., Gradient Boosted Decision Tree (GBDT)): reduces bias by focusing on mistakes made by previous models
- GBDT is one of the most powerful and widely used machine learning models, particularly well suited for tabular data

References

fixme

- fixme



References i

scikit-learn developers.
4.2. permutation feature importance.
https://scikit-learn.org/stable/modules/permutation_importance.html, 2025.
Accessed: 2025-06-01.

Sirakorn.
Illustration of a boosting method for ensemble learning — Wikipedia, the free encyclopedia, 2020.
Accessed: 2025-06-01.

References ii

Sirakorn.
Illustration of a bootstrap aggregating (bagging) method for ensemble learning — Wikipedia, the free encyclopedia, 2020.
Accessed: 2025-06-01.